CodeAlpha Python Internship Tasks

Task 1: Hangman Game

This script implements a simple, text-based Hangman game. A word is chosen randomly from a predefined list, and the player has 6 incorrect attempts to guess it letter by letter.

hangman_game.py

Python

```python
import random

def hangman():
    """
    A simple text-based Hangman game.
    The player has 6 incorrect guesses to guess the word.
    """
    # A small list of 5 predefined words as per the instructions.
    word_list = ["python", "alpha", "code", "intern", "developer"]
    secret_word = random.choice(word_list).lower()
    guessed_letters = []
    incorrect_guesses = 0
    max_incorrect_guesses = 6 # Limit incorrect guesses to 6.

    print("Welcome to Hangman!")
    print("I'm thinking of a word. You have 6 attempts to guess it.")

    # The main game loop continues as long as the player has guesses left.
    while incorrect_guesses < max_incorrect_guesses:
        # Display the current state of the word (e.g., "_ p p _ e")
        display_word = ""
        for letter in secret_word:
            if letter in guessed_letters:
                display_word += letter + " "
            else:
                display_word += "_ "
        print(f"\nWord: {display_word}")
```

```
        print(f"Guessed letters: {', '.join(guessed_letters)}")
        print(f"Incorrect guesses left: {max_incorrect_guesses - incorrect_guesses}")

        # Check if the player has won
        if "_" not in display_word:
            print("\nCongratulations! You guessed the word correctly!")
            print(f"The word was: {secret_word}")
            break

        # Get player input.
        guess = input("Guess a letter: ").lower()

        # --- Input Validation ---
        if not guess.isalpha() or len(guess) != 1:
            print("Invalid input. Please enter a single letter.")
            continue

        if guess in guessed_letters:
            print("You have already guessed that letter. Try another one.")
            continue

        guessed_letters.append(guess)

        # --- Check the Guess ---
        if guess in secret_word:
            print(f"Good guess! '{guess}' is in the word.")
        else:
            incorrect_guesses += 1
            print(f"Sorry, '{guess}' is not in the word.")

    # After the loop, if the player has run out of guesses, they lose.
    else:
        print("\n--- Game Over ---")
        print("You ran out of guesses.")
        print(f"The secret word was: {secret_word}")

# To run the game
if __name__ == "__main__":
    hangman()
```

Task 2: Stock Portfolio Tracker

This script allows a user to input their stock holdings and calculates the total value of their portfolio based on a predefined set of prices. It also offers to save the portfolio to a text file.

portfolio_tracker.py

Python

```
def stock_portfolio_tracker():
    """
    A simple stock portfolio tracker that calculates the total investment value
    based on user input and a hardcoded dictionary of stock prices.
    """
    # Hardcoded dictionary to define stock prices.
    stock_prices = {
        "AAPL": 180.00,
        "TSLA": 250.00,
        "GOOGL": 140.00,
        "MSFT": 330.00,
        "AMZN": 135.00
    }

    portfolio = {}
    total_investment_value = 0

    print("Welcome to the Stock Portfolio Tracker.")
    print("Available stocks and their prices:")
    for stock, price in stock_prices.items():
        print(f"- {stock}: ${price:.2f}")

    while True:
        # User inputs stock name.
        stock_symbol = input("\nEnter a stock symbol to add to your portfolio (or 'done' to finish):
").upper()

        if stock_symbol == 'DONE':
            break

        if stock_symbol not in stock_prices:
            print("Invalid stock symbol. Please choose from the available stocks.")
            continue
```

```
        # User inputs stock quantity.
        try:
            quantity = int(input(f"Enter the quantity for {stock_symbol}: "))
            if quantity <= 0:
                print("Quantity must be a positive number.")
                continue
        except ValueError:
            print("Invalid quantity. Please enter a whole number.")
            continue

        portfolio[stock_symbol] = portfolio.get(stock_symbol, 0) + quantity
        print(f"Added {quantity} shares of {stock_symbol} to your portfolio.")

    # --- Calculate and Display Total Value ---
    print("\n--- Your Stock Portfolio ---")
    if not portfolio:
        print("Your portfolio is empty.")
    else:
        for symbol, qty in portfolio.items():
            price = stock_prices[symbol]
            value = price * qty
            total_investment_value += value
            print(f"{symbol}: {qty} shares @ ${price:.2f} = ${value:.2f}")

        print("-----------------------------")
        print(f"Total Portfolio Value: ${total_investment_value:.2f}")

        # Optional: Save the result in a .txt file.
        save_file = input("\nDo you want to save your portfolio to a file? (yes/no): ").lower()
        if save_file == 'yes':
            with open("portfolio.txt", "w") as file:
                file.write("--- Your Stock Portfolio ---\n")
                for symbol, qty in portfolio.items():
                    price = stock_prices[symbol]
                    value = price * qty
                    file.write(f"{symbol}: {qty} shares @ ${price:.2f} = ${value:.2f}\n")
                file.write("-----------------------------\n")
                file.write(f"Total Portfolio Value: ${total_investment_value:.2f}\n")
            print("Portfolio saved to 'portfolio.txt'.")

# To run the tracker
if __name__ == "__main__":
```

```
stock_portfolio_tracker()
```

Task 3: Task Automation - Email Extractor

This script automates the task of extracting all email addresses from a given text file (input.txt) and saving them to a new file (extracted_emails.txt).

Instructions:

1. Create a file named input.txt in the same directory as the Python script.
2. Add some text containing various email addresses into input.txt.
3. Run the script. A new file named extracted_emails.txt will be created with the results.

email_extractor.py

Python

```
import re # Using the 're' module for regular expressions.

def extract_emails_from_file(input_filename="input.txt", output_filename="extracted_emails.txt"):
    """
    Extracts all email addresses from a .txt file and saves them to another file.
    """
    # A robust regex for matching standard email addresses.
    email_regex = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"

    try:
        # Reading from the input file.
        with open(input_filename, 'r') as f_in:
            content = f_in.read()

        # Find all matching email addresses in the content.
        found_emails = re.findall(email_regex, content)

        if not found_emails:
            print(f"No email addresses were found in '{input_filename}'.")
            return

        # Writing the extracted emails to the output file.
        with open(output_filename, 'w') as f_out:
```

```
            for email in found_emails:
                f_out.write(email + '\n')

        print(f"Successfully extracted {len(found_emails)} email(s).")
        print(f"Results have been saved to '{output_filename}'.")

    except FileNotFoundError:
        print(f"Error: The file '{input_filename}' was not found.")
        print("Please create the file and add some text with emails to it.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

# To run the script
if __name__ == "__main__":
    extract_emails_from_file()
```

Task 4: Basic Chatbot

This script creates a simple, rule-based chatbot that provides predefined replies to specific user inputs like "hello," "how are you," or "bye."

chatbot.py

Python

```
def simple_chatbot():
    """
    A simple rule-based chatbot that responds to predefined user inputs.
    The chatbot operates in a loop until the user says 'bye'.
    """
    print("Chatbot: Hi! I am a simple chatbot. You can talk to me.")
    print("Chatbot: Type 'bye' to exit.")

    # The main loop for the chat session.
    while True:
        # Get input from the user.
        user_input = input("You: ").lower().strip()

        # Rule-based responses using if-elif-else.
        if user_input == "hello" or user_input == "hi":
            print("Chatbot: Hi there!") # Predefined reply.
```

```
        elif "how are you" in user_input:
            print("Chatbot: I'm just a bot, but I'm doing great! How can I help you today?") # Predefined reply.
        elif user_input == "bye":
            print("Chatbot: Goodbye! Have a nice day.") # Predefined reply.
            break # Exit the loop.
        elif "your name" in user_input:
            print("Chatbot: I am a simple rule-based chatbot created for a CodeAlpha task.")
        elif "help" in user_input:
            print("Chatbot: You can ask me 'how are you' or say 'hello'. Type 'bye' to end our chat.")
        else:
            print("Chatbot: Sorry, I don't understand that. Please try asking something else.")

# To run the chatbot
if __name__ == "__main__":
    simple_chatbot()
```